



Southeast University

Department of Computer Science & Engineering

Complex Engineering Project (CEP)

Submitted by,

Md. Abdullah Al Afif

Course Code – CSE345

**Course Title – Information System Design &
Software Engineering**

Submitted to,

Mehedi Ahmed

Lecturer, Department of CSE

Southeast University

Table of Contents

1. Requirements Engineering and Stakeholder Analysis (Task 1)	3
1.1 Context and Approach	3
1.2 Requirement Categories	3
1.3 Stakeholder Analysis	4
1.4 Prioritization Framework	5
1.5 Trade-offs When Requirements Conflict	5
2. Structured Software Design Using UML (Task 2)	6
2.1 Design Approach	6
2.2 Use Case Diagram	6
2.3 Class Diagram	7
2.4 Sequence Diagram	8
2.5 Activity Diagram	9
3. Tools, Platforms, Technologies, and Methodology (Task 3)	11
3.1 Development Methodology	11
3.2 Technology and Platform Choices	11
3.3 How Each Choice Responds to the Scenario's Constraints	11
3.4 Acknowledged Uncertainty and Trade-offs	12
4. Project Timeline — Phase 1 Gantt Chart	13

1. Requirements Engineering and Stakeholder Analysis (Task 1)

1.1 Context and Approach

NadiPath Multimodal Logistics Ltd. must digitize an inland waterway freight network that is under heavy pressure after Cyclone Mahaan. The network now carries about 60% of inter-district freight, but it still runs on paper. The project also has hard limits: only 14 months, a budget of USD 8.5 million, weak internet at four terminals, an old 2009 vessel registry, three disconnected regulator platforms, and stakeholders who do not trust digitization. This section breaks the scenario into clear requirements and shows how they were prioritized.

1.2 Requirement Categories

Requirements are split into three groups so the design team and NadiPath leadership can track them separately: Functional Requirements (FR – what the system must do), Non-Functional Requirements (NFR – how well it must do it), and Organizational/Regulatory Requirements (OR – rules imposed by law, agencies, or governance). Splitting them this way makes it easier to see which requirements come from a technical need and which come from a stakeholder conflict.

1.2.1 Functional Requirements

ID	Requirement	Primary Stakeholder	Priority
FR1	Register vessels and link them to the existing 2009 registry records	BIWTA, Vessel Operators	Must
FR2	Create and manage voyage schedules across 11 terminals	Vessel Operators, BIWTA	Must
FR3	Allow traders (including small/informal traders) to submit cargo manifests digitally	Informal Traders	Must
FR4	Route clearance requests to NBR, BIWTA, and BSTI and collect their decisions	Regulatory Bodies	Must
FR5	Queue transactions locally at terminals with weak or no connectivity and sync later	Terminal Operators	Must
FR6	Assign and track dock workforce shifts linked to vessel arrivals	Dock Workers, BIWTA	Should
FR7	Provide disaster/reroute advisories from the Department of Disaster Management	Disaster Mgmt. Dept.	Should
FR8	Generate ISO-auditable compliance and financial reports for the World Bank	NadiPath Leadership, World Bank	Must
FR9	Provide a simple, low-literacy-friendly interface (SMS/USSD fallback)	Informal Traders, Dock Workers	Should
FR10	Allow role-based manual override during disputes or system downtime	BIWTA Officers	Could

1.2.2 Non-Functional Requirements

ID	Requirement	Reason	Priority
NFR1	System must work offline and sync automatically once a connection returns	4 terminals have intermittent connectivity	Must
NFR2	Data exchange with legacy systems must not require replacing them	2009 registry cannot be replaced in this timeline	Must
NFR3	All actions must be logged for an ISO audit trail	World Bank funding is conditional on an auditable platform	Must
NFR4	The interface must be usable by people with low digital literacy	Small traders and dock workers have limited tech exposure	Should
NFR5	The platform must scale to 11 terminals within budget	USD 8.5 million, 14-month limit	Must
NFR6	Sensitive competitor data must be isolated between private operators	Operators resist sharing data with rivals	Must
NFR7	The system must be introduced gradually with visible worker benefits	Two earlier digitization attempts were resisted by dock workers	Must

1.2.3 Organizational / Regulatory Requirements

ID	Requirement	Priority
OR1	Comply with data-handling rules of NBR, BIWTA, and BSTI without forcing them to change their own systems immediately	Must
OR2	Protect small traders from sudden full regulatory exposure while still improving traceability	Should
OR3	Provide governance structure across the consortium (BIWTA, 3 private shippers, DDM, sovereign fund)	Must
OR4	Meet World Bank procurement, safeguard, and audit conditions tied to the USD 120 million financing	Must

1.3 Stakeholder Analysis

The scenario has stakeholders with very different, and sometimes opposing, interests. The table below summarizes each group's interest, influence over the project, and where their interest conflicts with another group.

Stakeholder	Interest	Influence	Main Conflict
BIWTA	Regain control and visibility over the network	High	Wants central data; operators resist sharing
Private shipping operators (3)	Keep competitive data private, minimize new costs	High	Resist sharing schedule/cargo data with rivals
Small/informal traders	Keep working with minimal paperwork and cost	Low-Medium	Fear that digitization exposes them to tax/regulatory scrutiny
Dock workers	Job security, fair shift allocation	Medium (can halt operations)	Fear automation will replace or reduce their roles
NBR / BSTI (regulators)	Full compliance data, tax and standards enforcement	High	Want more data than traders/operators want to give
Department of Disaster Management	Fast rerouting and safety data during emergencies	Medium	Needs real-time data even from offline terminals

Stakeholder	Interest	Influence	Main Conflict
Sovereign investment fund	Return on investment, project viability	High	Wants speed; some fixes (trust-building) need time
World Bank (financier)	ISO-auditable, transparent, on-schedule delivery	Very High	Compliance requirements can slow adoption-friendly rollout

1.4 Prioritization Framework

Requirements were prioritized using a combined MoSCoW and Criticality–Feasibility method. First, each requirement was labelled Must/Should/Could/Won't (MoSCoW) based on whether the project fails its core goal without it. Second, every "Must" requirement was plotted against two scores from 1 (low) to 5 (high): Criticality (how directly it fixes a core network failure, such as freight visibility or clearance delay) and Feasibility (how realistic it is within the 14-month, USD 8.5 million, and connectivity limits). Requirements with high criticality and high feasibility (for example FR1, FR3, FR4, NFR1, NFR2) were placed in the Phase 1 scope. Requirements with high criticality but lower near-term feasibility (for example full automation of dock workforce management, FR6) were scheduled for a later phase.

This framework was chosen over a single MoSCoW list because MoSCoW alone does not show why one "Must" requirement should be built before another "Must" requirement. Adding feasibility scoring gives NadiPath's leadership a defensible order for a project that cannot do everything at once.

1.5 Trade-offs When Requirements Conflict

Several requirements cannot all be satisfied at the same time, and the design has to accept a trade-off rather than pretend one does not exist.

- Transparency vs. privacy: Regulators (NBR, BSTI) want full visibility into cargo and pricing data, but private operators do not want competitors to see this data even indirectly. The trade-off chosen is role-based data segregation: regulators see compliance-relevant fields only, not full commercial data such as pricing between operators.
- Formal compliance vs. informal trader participation: Full digitization could expose small traders to taxes or fees they were previously avoiding, which may push them back to informal, undocumented movement of goods. The trade-off is a simplified, lower-friction registration tier for small traders in Phase 1, with full compliance features phased in later once trust is built.
- Speed vs. worker trust: The sovereign fund and World Bank want fast delivery, but dock workers have already stopped two digitization attempts. The trade-off is to spend part of the 14-month schedule (see Gantt chart, Section 4) on visible worker benefits (fair shift assignment, no job cuts in Phase 1) even though this slows pure technical delivery.
- Central control vs. terminal autonomy: BIWTA wants centralized scheduling, but four terminals cannot always reach the central platform. The trade-off is the offline-first design (NFR1), which gives terminals local autonomy during outages at the cost of short-term data lag until sync happens.
- Audit completeness vs. legacy constraints: Full ISO auditability is easiest with a single unified database, but the 2009 registry and three regulator platforms cannot be replaced. The trade-off is an adapter-based integration layer (see the adapter design in Section 2.4) that keeps legacy systems running while still producing a unified audit trail.

2. Structured Software Design Using UML (Task 2)

2.1 Design Approach

The design follows an offline-first, adapter-based architecture. Every terminal has a local edge node that can keep working without internet, and legacy systems (the 2009 registry, NBR, BIWTA legacy records, BSTI) are reached only through translator adapters, so they never need to be replaced. Four UML diagram types were selected because together they cover structure, behavior, and physical deployment, which is enough to explain both the software and the constraint-driven architecture: a Use Case Diagram (who does what), a Class Diagram (data structure), a Sequence Diagram (step-by-step interaction), an Activity Diagram (business workflow and decisions).

2.2 Use Case Diagram

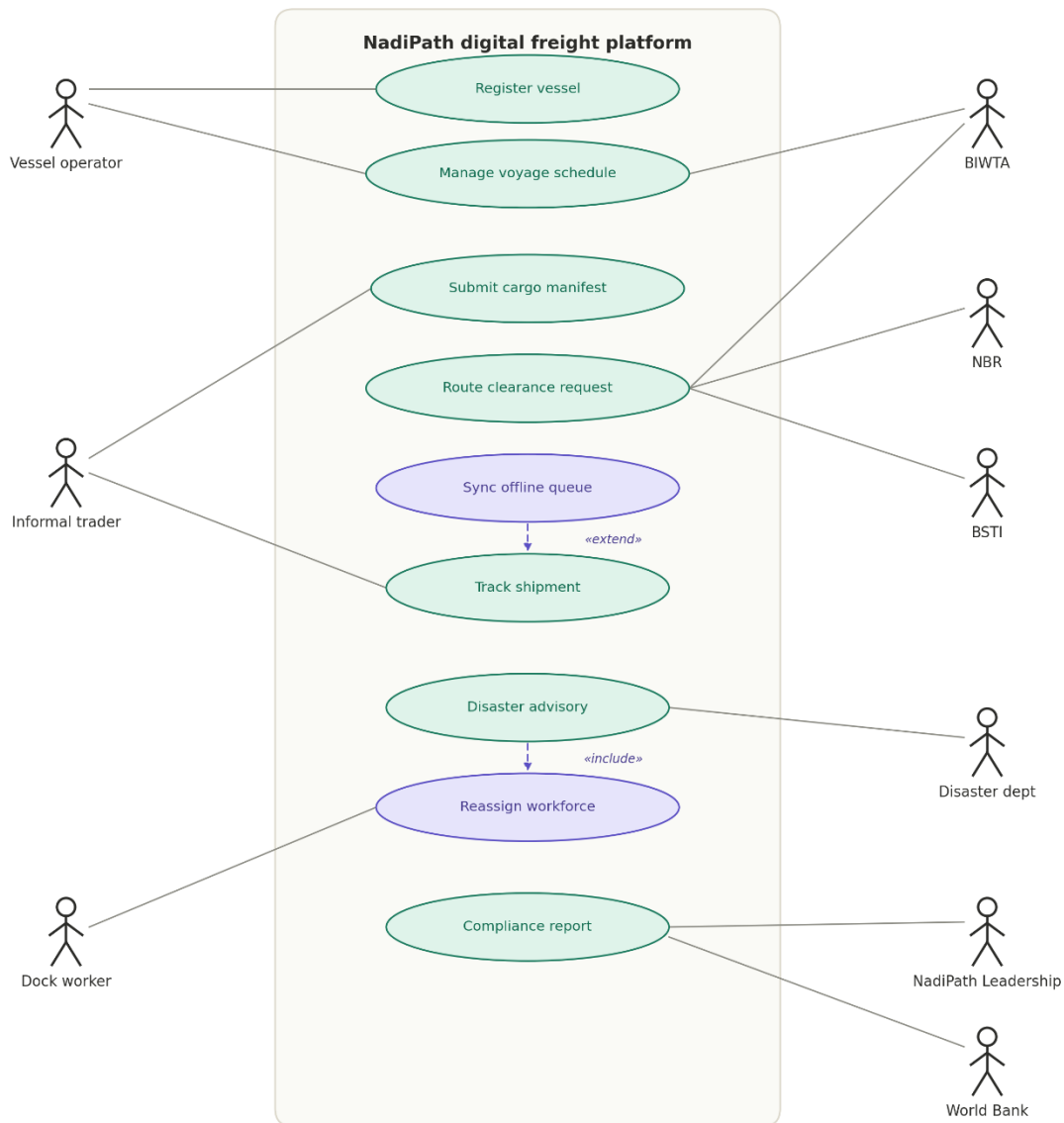


Figure 2.1 — Use Case Diagram of the NadiPath Digital Freight Platform

Commentary

Actors were split by real conflict of interest rather than grouped generically. "Small Informal Trader" is kept separate from "Vessel Operator" because traders have a very different relationship with regulation (see Section 1.5), and merging them into one generic "Customer" actor, which was considered and rejected, would have hidden that conflict from the design. The "Sync Offline Transaction Queue" use case exists only because of the intermittent-connectivity constraint; a simpler design without this use case was considered first but rejected because it would break at the four low-connectivity terminals. The "<<extend>>" relation from Sync to Track Shipment shows that syncing is an optional path that only runs when a terminal was previously offline. The "<<include>>" relation shows that a disaster advisory always triggers the workforce reassignment use case, since a rerouted vessel needs a new dock/workforce plan.

2.3 Class Diagram

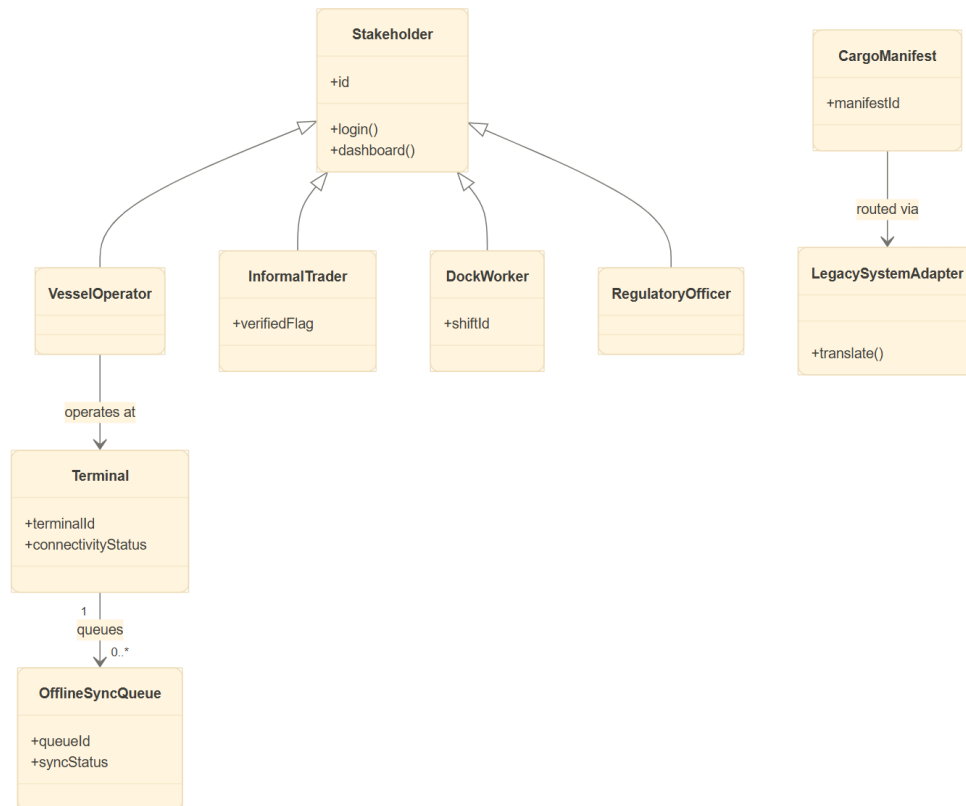


Figure 2.2 — Class Diagram of core domain entities

Commentary

An abstract Stakeholder class was used as the parent of VesselOperator, InformalTrader, DockWorker, and RegulatoryOfficer because all four share login and dashboard behavior, but each needs its own data (for example, a trader needs a verified flag, while a dock worker needs a shift ID). A flatter design with one single "User" class and a role field was considered, but it was rejected because it would force every business rule about a role into if/else checks in code instead of the class structure, which becomes hard to maintain once more roles are added later. The OfflineSyncQueue and LegacySystemAdapter classes are included directly in the domain model, not hidden as "technical detail," because they are core to solving two of the scenario's hardest constraints: connectivity and legacy integration. The relationship between Terminal and OfflineSyncQueue shows that queuing happens per terminal, matching the fact that only some of the 11 terminals have connectivity problems.

2.4 Sequence Diagram

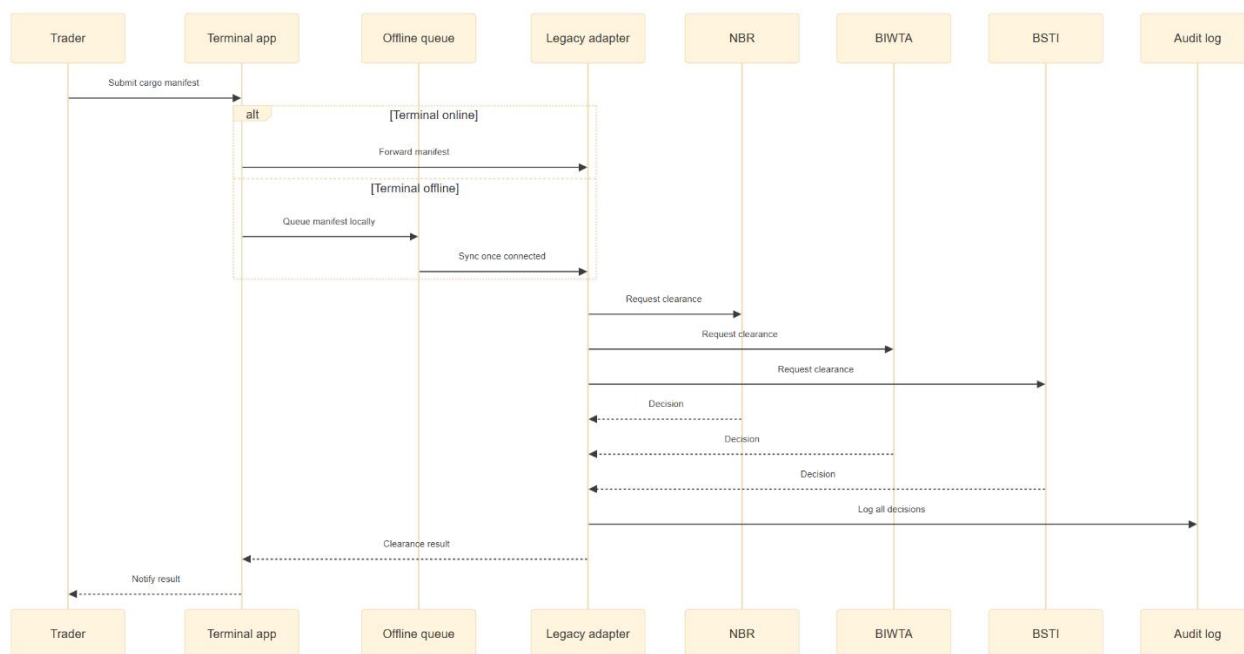


Figure 2.3 — Sequence Diagram: cargo manifest submission and cross-agency clearance

Commentary

This scenario was chosen because it touches almost every hard constraint at once: a small trader with limited connectivity, a legacy adapter translating to three different regulator systems, and an audit-log requirement. The diagram shows two paths at step 2: a direct forward when the terminal is online, and a queued path when it is not. An alternative design was considered where the mobile app talks directly to each regulator system; this was rejected because it would require the trader's app to understand three different legacy formats (NBR, BIWTA, BSTI), which is fragile and expensive to maintain. Centralizing translation in one Legacy Adapter service means only that one component changes if a regulator updates their system.

2.5 Activity Diagram

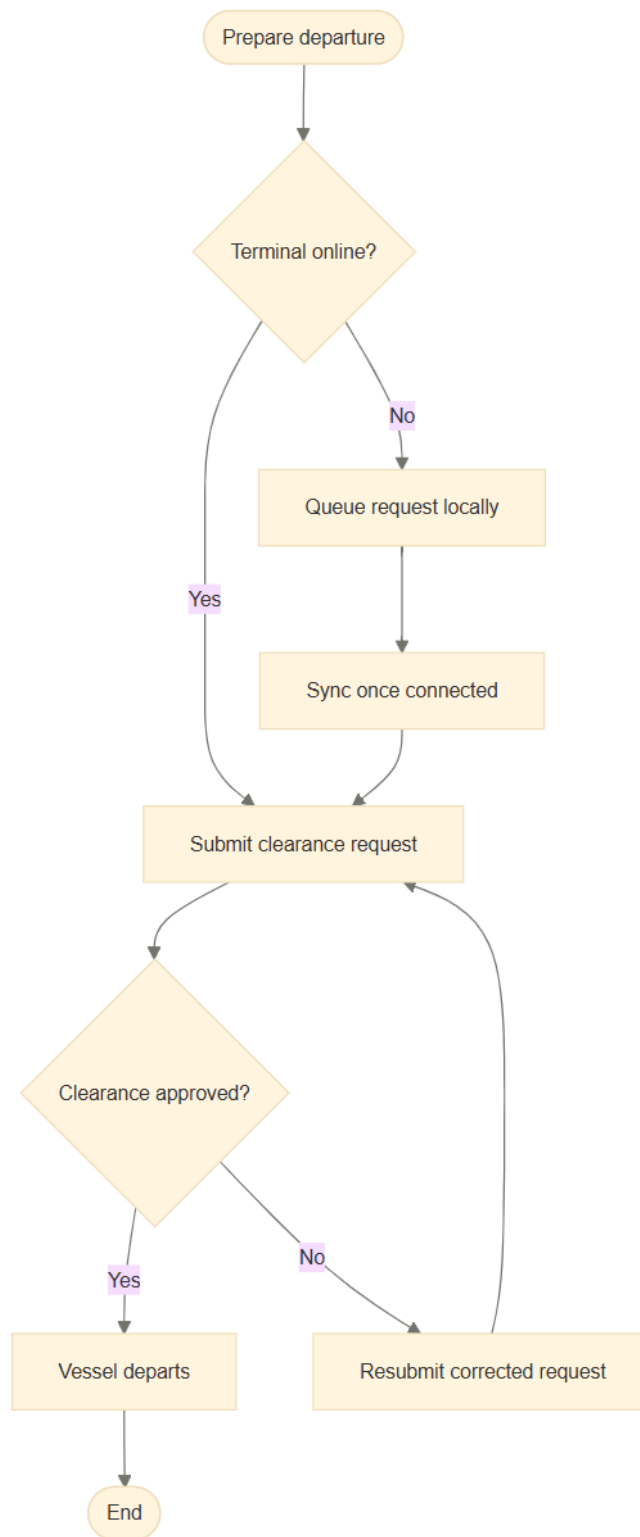


Figure 2.4 — Activity Diagram: vessel departure and clearance workflow

The two decision points in this diagram were placed deliberately to match the two constraints that most often break a paper-based process today: network connectivity and multi-agency approval. Without the connectivity

decision, the workflow would stop completely at any of the four weak-connectivity terminals; the queue-and-sync path keeps work moving locally. Without the clearance decision, there would be no clear path for what happens when a request is rejected, which is common today and causes delays; the diagram gives a defined resubmission loop instead of leaving rejected cargo stuck with no next step.

3. Tools, Platforms, Technologies, and Methodology (Task 3)

3.1 Development Methodology

A hybrid Agile methodology is recommended: short two-to-three-week Scrum sprints for building software features, combined with fixed regulatory and stakeholder review gates that do not move even if a sprint runs late. A pure Agile/Scrum approach was considered but rejected as the only method, because regulatory sign-off from NBR, BIWTA, and BSTI, and World Bank audit checkpoints, cannot be treated as flexible backlog items; they are hard external gates. A pure Waterfall approach was also rejected, because the scenario is open-ended and multidisciplinary, and requirements around trust-building with dock workers are expected to change as the pilot terminals give feedback. The hybrid model lets the technical team iterate quickly while still giving NadiPath's leadership and the World Bank fixed milestones to audit against (see the Gantt chart in Section 4).

3.2 Technology and Platform Choices

Layer	Chosen Technology	Why
Edge / terminal client	Progressive Web App + lightweight Android app, with local SQLite storage	Works on low-cost devices, keeps working offline, avoids app-store update delays at remote terminals
Low-literacy access channel	SMS / USSD gateway	Reaches small traders and dock workers without smartphones or strong data literacy
Core backend services	Java (Spring Boot) or Node.js microservices	Mature, open-source, large talent pool in Bangladesh, keeps licensing cost low
Central database	PostgreSQL	Open-source, reliable, strong support for structured audit and compliance data
Offline sync mechanism	Queue-based sync with conflict resolution rules (last-write-wins with manual review flag for conflicts)	Matches the intermittent-connectivity constraint without needing a fully distributed database
Integration / legacy layer	Dedicated adapter microservices + a message queue (e.g., RabbitMQ)	Isolates the 2009 registry and NBR/BIWTA/BSTI platforms so they never need to be replaced
Hosting	Government-approved local data center with cloud burst capability (hybrid cloud)	Meets data-sovereignty and regulatory expectations while allowing scaling
Security & audit	Role-based access control, encryption in transit and at rest, immutable audit logging	Required for the ISO-auditable condition tied to World Bank financing
Project tracking	Open-source Scrum board (e.g., self-hosted Jira alternative) + shared Gantt tracking	Keeps recurring software cost low, matches hybrid Agile methodology

3.3 How Each Choice Responds to the Scenario's Constraints

Constraint	Response	Risk Introduced / Mitigated
Budget (USD 8.5M, 14 months)	Open-source backend, database, and messaging stack; phased rollout instead of building all 11 terminals at once	Mitigates: license and hardware cost. Introduces: reliance on in-house/contracted skill for open-source support instead of vendor support contracts
Intermittent connectivity (4 terminals)	Offline-first edge nodes with local storage and queued sync	Mitigates: work stoppage during outages. Introduces: short-term data lag and the need for conflict-resolution rules when two terminals edit related data while both offline
2009 legacy registry	Read-only API wrapper around the legacy registry instead of replacing it	Mitigates: cost/time of migrating 15+ years of vessel records. Introduces: dependency on the old system's uptime and data quality
Disconnected NBR/BIWTA/BSTI platforms	Dedicated legacy adapters per agency, translating each one's own format	Mitigates: need for agencies to change their own systems. Introduces: adapter maintenance cost if an agency changes its format without notice
Private operator data-sharing resistance	Role-based data segregation; regulators and competitors only see the fields they are entitled to	Mitigates: operator refusal to participate. Introduces: extra design and testing effort for fine-grained permissions
Dock worker resistance (2 earlier failed attempts)	Phased rollout, visible workforce benefits first, training built into the schedule (Section 4)	Mitigates: work stoppage/boycott. Introduces: slower pure technical delivery than a big-bang rollout
Small trader fear of regulatory exposure	Simplified low-friction registration tier in Phase 1	Mitigates: traders avoiding the platform entirely. Introduces: a temporary gap in full compliance data until later phases

3.4 Acknowledged Uncertainty and Trade-offs

Some choices are not risk-free and are stated openly rather than hidden:

- Progressive Web App vs. native app: A PWA is cheaper and easier to update at remote terminals, but it may perform worse on very old Android devices than a native app would. This is accepted because budget and update-speed matter more than raw performance for Phase 1.
- Open-source stack vs. vendor-supported platform: Open-source tools reduce license cost but shift support risk onto NadiPath's own technical team or a support contract, which must be budgeted for beyond Phase 1.
- Queue-based sync vs. complex distributed database: A queue-based sync is simpler to build in 14 months but is not a perfect real-time system; short data lag at offline terminals is a real and accepted limitation, not something the design pretends to solve completely.
- Phased rollout vs. World Bank's preference for fast, visible results: Spending early months on trust-building with dock workers (see Gantt chart) may look slower on paper, but the alternative — repeating the two earlier failed digitization attempts — carries a higher risk of total project failure.

4. Project Timeline — Phase 1 Gantt Chart

The chart below lays out the 14-month, USD 8.5 million Phase 1 delivery. It reflects the trade-offs explained in Sections 1 and 3: trust-building with dock workers and traders runs in parallel with technical work from month 1, rather than being left until the end, and the rollout to all 11 terminals happens in a phased way after a 2-terminal pilot proves the offline-sync and legacy-adaptor design works.

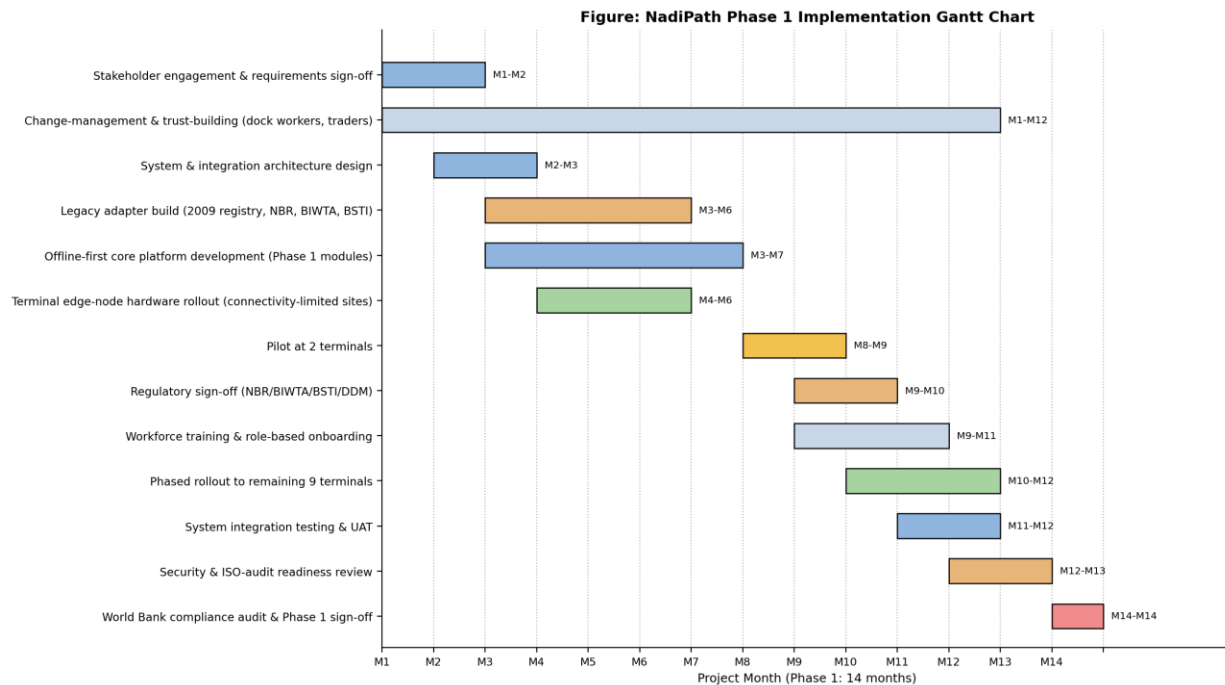


Figure 4.1 — NadiPath Phase 1 Implementation Gantt Chart (14 months)

Key sequencing decisions: the legacy adapter build (months 3–6) starts only after the architecture design (months 2–3) is complete, since the adapters depend on a stable integration design. The pilot (months 8–9) is deliberately placed before the full 9-terminal rollout (months 10–12) so that lessons from real dock workers and traders at two terminals can be applied before scaling. The World Bank compliance audit is placed at the very end (month 14) after the ISO-audit readiness review (months 12–13), to leave a buffer for fixing any audit findings within the 14-month limit.